

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте

на тему Визуализация алгоритмов машинного обучения
Итоговый отчёт по курсовому проекту

Выполнен студентом:

Группы #БПМИ2412-2

Новолодский Игорь Олегович

Проверен руководителем проекта:

Самоненко Илья Юрьевич
доцент
НИУ ВШЭ

Содержание

1	Аннотация	2
2	Постановка задачи	2
3	Решающее дерево	3
3.1	Идея	3
3.2	Что хранится в листе	3
3.3	Как дерево выбирает вопрос	4
4	Почему одного дерева обычно мало	4
5	Случайный лес	5
5.1	Идея	5
5.2	Bootstrap	6
5.3	Случайность в направлениях	6
6	Косые деревья решений	7
6.1	В чём отличие от обычного дерева	7
6.2	Как это устроено внутри	7
7	Как устроено веб-приложение	9
7.1	Путь данных внутри сайта	9
8	Метрики качества	12
9	Jupyter Notebook	13
10	Как пользоваться материалами	15
11	Ограничения проекта	15
12	Заключение	15
13	Список литературы	16

1 Аннотация

Этот проект про три алгоритма классификации: обычное решающее дерево, случайный лес и их менее известного родственника, oblique-дерево, у которого разбиения идут не по прямым осям, а под произвольным углом. Разбираться в них на таблицах с абстрактными признаками скучно и не очень наглядно, поэтому я взял задачу, которую видно глазами: пользователь рисует чёрно-белую картинку на холсте, а модель пытается угадать, какие точки чёрные, а какие белые, зная только их координаты. Каждый пиксель картинки превращается в объект с двумя признаками, x и y , и меткой класса: 1, если он закрашен, 0, если это фон.

Такая постановка задачи хороша тем, что ошибки модели не нужно искать в столбце ассигасу: они сразу видны на картинке. Обычное дерево зачастую рисует грубые прямоугольные заплатки вместо плавных линий. Случайный лес выглядит заметно аккуратнее: он усредняет десятки таких деревьев и сглаживает случайные огрехи каждого из них. А oblique-дерево умеет резать плоскость наклонными линиями, поэтому диагональные штрихи и дуги ему даются гораздо легче.

Работа состоит из трёх частей: веб-приложения на чистом JavaScript, короткого Jupyter Notebook со scikit-learn и этого текста. Смотреть их лучше вместе: здесь объясняются идеи, в приложении их можно потрогать руками, подвигать ползунки и нарисовать что-то своё, а notebook показывает, как похожая логика выглядит на обычном Python.

2 Постановка задачи

Машинное обучение любят объяснять на табличках: есть строки, есть признаки, есть ответ. Классический пример: предсказать цену квартиры по её площади и расстоянию до метро. В этом проекте таблица тоже есть, только вместо квартир там пиксели.

Возьмём холст размером 256×256 . Координаты каждого пикселя нормируются в отрезок:

$$x \in [0, 1], \quad y \in [0, 1].$$

Метка класса определяется тем, что нарисовал пользователь:

$$label = \begin{cases} 1, & \text{пиксель относится к рисунку,} \\ 0, & \text{пиксель относится к фону.} \end{cases}$$

Дальше всё сводится к обычной задаче бинарной классификации:

$$f(x, y) \rightarrow \{0, 1\}.$$

По координатам точки модель должна решить, чёрная она или белая.

Здесь стоит сразу проговорить одну тонкость: модель не хранит внутри себя копию картинки, она видит только координаты точек и их метки и по ним строит правило. Потом это правило прогоняется по всем 256×256 пикселям холста заново, и из ответов собирается новое изображение. Каждый алгоритм ошибается по-своему: у одного граница получается ступенчатой, у другого рваной, третий может смазать тонкую деталь или, наоборот, закрасить лишний кусок фона. Собственно, в этих различиях и есть весь интерес проекта.

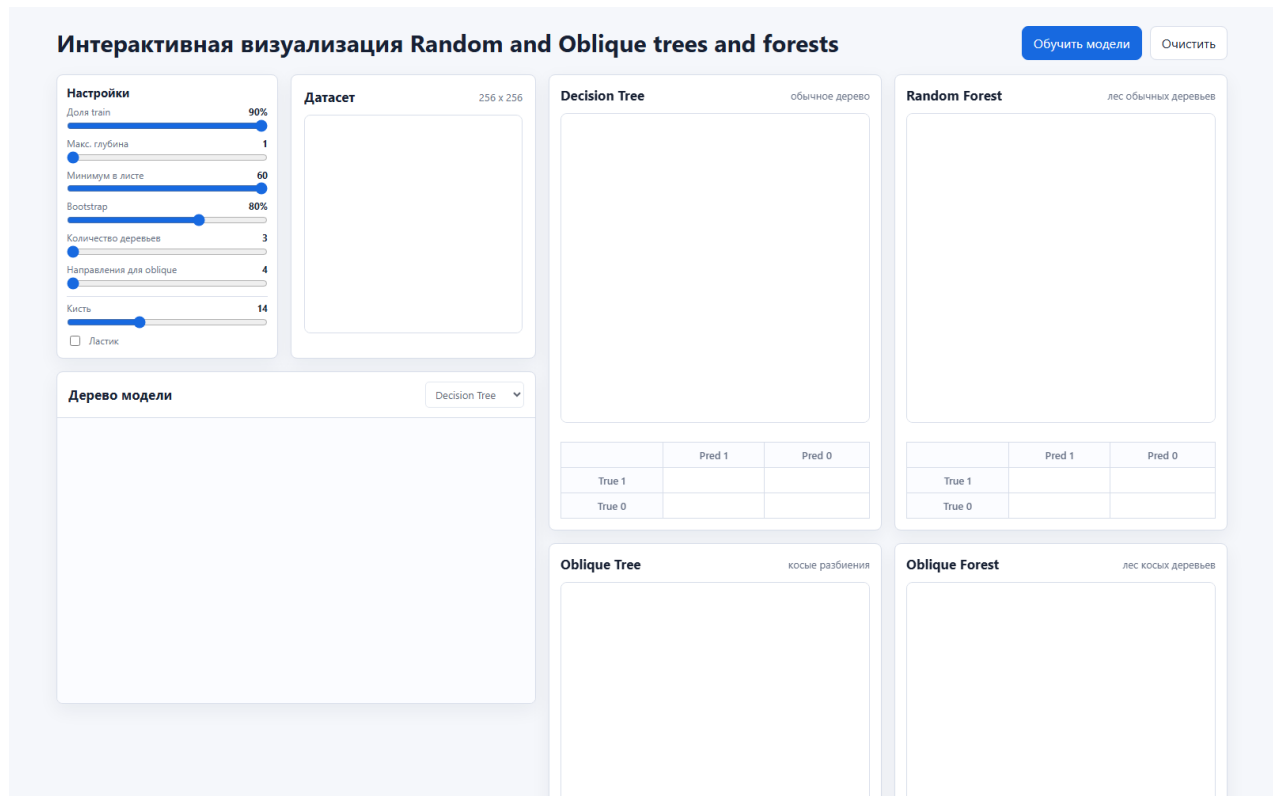


Рис. 1: Главный экран приложения: слева настройки, по центру холст, справа карточки с ответами моделей.

3 Решающее дерево

3.1 Идея

Решающее дерево работает как игра в вопросы. Модель смотрит на точку и задаёт вопрос, например:

$$x < 0.45?$$

Ответ «да» отправляет точку в левую ветку, «нет» в правую. Внутри каждой ветки задаётся следующий вопрос, и так по цепочке, пока точка не окажется в листе. Лист и выдаёт финальный вердикт: «рисунок» или «фон».

В обычном дереве вопросы устроены максимально просто:

$$x < c \quad \text{или} \quad y < c.$$

Геометрически это горизонтальные и вертикальные линии на плоскости. Значит, дерево умеет резать пространство только на прямоугольники. С одной стороны, это удобно: правило легко читается и объясняется. С другой стороны, диагональную линию таким образом приходится собирать из кучи мелких прямоугольных ступенек, и результат выглядит соответствующе.

3.2 Что хранится в листе

В каждом узле дерева лежит какой-то кусок обучающих точек. Если среди них преобладают чёрные пиксели, узел, скорее всего, относится к рисунку, если белые, то к фону. Формально в листе хранится доля точек класса 1:

$$p = \frac{\text{число чёрных точек в листе}}{\text{число всех точек в листе}}.$$

Чтобы получить чёрно-белую картинку, а не размытые оттенки серого, эта доля округляется порогом 0,5:

$$\hat{y} = \begin{cases} 1, & p \geq 0.5, \\ 0, & p < 0.5. \end{cases}$$

3.3 Как дерево выбирает вопрос

Дерево перебирает возможные разбиения и ищет то, после которого точки в двух получившихся группах становятся более однородными по классу. Мера этой однородности называется индексом Джини:

$$G = 1 - p^2 - (1 - p)^2,$$

где p это доля чёрных точек в узле. Если классы перемешаны примерно поровну, G близок к максимуму. Если в узле почти одни чёрные или почти одни белые точки, G стремится к нулю.

После разбиения на левую и правую части считается взвешенная нечистота:

$$G_{split} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R,$$

а выигрыш от разбиения показывает, насколько нечистота упала по сравнению с родительским узлом:

$$gain = G_{parent} - G_{split}.$$

Из всех вариантов дерево берёт тот, где $gain$ максимален.

4 Почему одного дерева обычно мало

У одного дерева есть классическая проблема: баланс между «слишком грубо» и «слишком подробно». Небольшая глубина означает мало вопросов и очень крупные, размашистые «мазки». Большая глубина даёт много деталей, но заодно дерево начинает подстраиваться под случайный шум конкретной выборки, а не под саму форму рисунка.

На картинке это заметно сразу: там, где пользователь провёл плавную дугу или диагональ, дерево вместо неё строит пиксельную лестницу из прямоугольников. Это не баг реализации, а прямое следствие того, как устроены его вопросы: только по одной координате за раз.

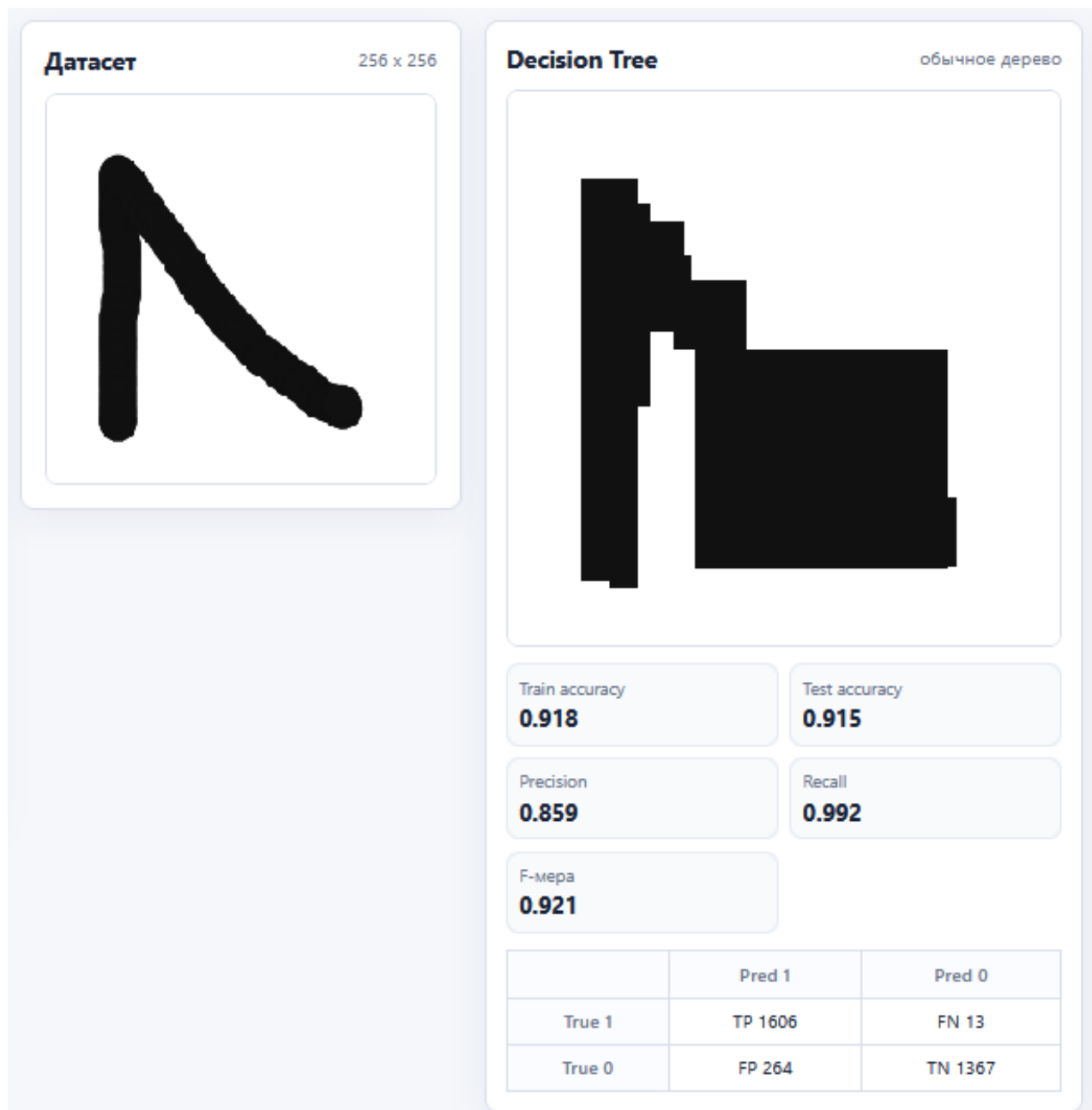


Рис. 2: Ответ обычного дерева: форма собирается вертикальными и горизонтальными разбиениями.

5 Случайный лес

5.1 Идея

Случайный лес это просто много решающих деревьев, работающих сообща. Каждое из них немного отличается от остальных, а итоговый ответ получается усреднением их предсказаний:

$$score(x, y) = \frac{1}{T} \sum_{i=1}^T tree_i(x, y),$$

где T это число деревьев в лесу.

Если одно дерево ошиблось из-за неудачного разбиения где-то в глубине, у остальных есть шанс эту ошибку компенсировать. В сумме лес получается устойчивее одного дерева, примерно как средняя оценка жюри надёжнее оценки одного судьи.

5.2 Bootstrap

Чтобы деревья в лесу не были клонами друг друга, каждое обучается на своей случайной подвыборке. Это bootstrap-выборка, собранная с возвращением: одна и та же точка может попасть в неё несколько раз, а другая не попасть вовсе.

В приложении за это отвечает ползунок **Bootstrap** со значениями от 40% до 100% от размера train-набора. При значении 80% (стоит по умолчанию) для каждого дерева набирается случайное множество точек размером 0,8 от обучающей выборки. В результате деревья видят слегка разные наборы данных и ошибаются в разных местах, а не одинаково.

5.3 Случайность в направлениях

В классическом случайном лесе случайность есть не только в объектах, но и в признаках: в каждом узле дерево смотрит не на все признаки сразу, а на случайную их часть. Здесь признаков всего два, x и y , поэтому для обычного леса дерево в узле иногда «видит» только одну из двух координат.

Для oblique-леса эта же идея переносится на наклонные направления. При поиске разбиения дерево перебирает не все возможные углы подряд, а случайно выбранную часть из них. Размер этой части задаёт ползунок **Направления** для oblique. Так деревья в oblique-лесу тоже получаются непохожими друг на друга, и усреднение работает как надо.

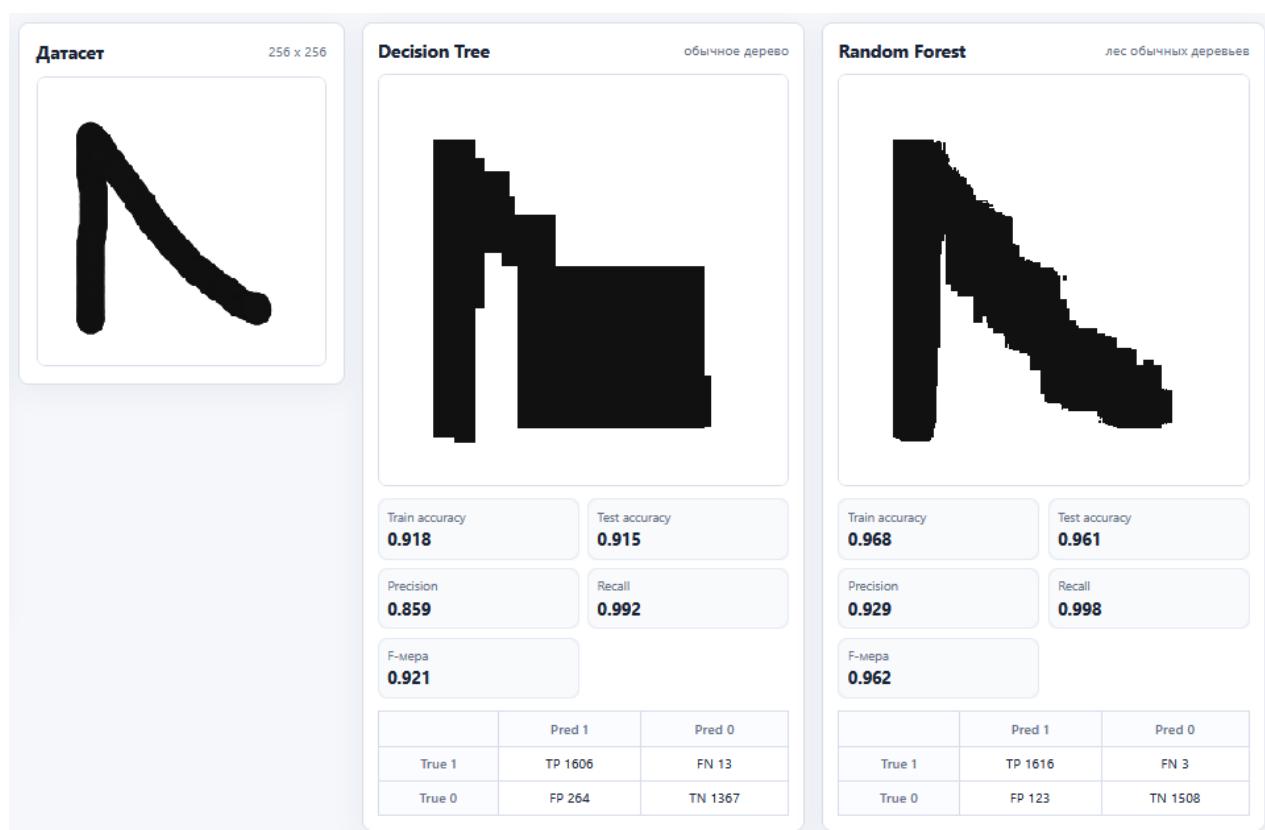


Рис. 3: Ответ случайного леса: много деревьев сглаживают ошибки друг друга.

6 Косые деревья решений

6.1 В чём отличие от обычного дерева

Обычное дерево в этой задаче умеет задавать вопросы только по одной координате:

$$x < c \quad \text{или} \quad y < c.$$

Косое дерево (oblique decision tree) идёт дальше и спрашивает про линейную комбинацию координат:

$$ax + by < c.$$

Геометрически это уже произвольная наклонная прямая, а не только вертикаль или горизонталь. Там, где обычному дереву нужна целая лестница из мелких прямоугольников, чтобы приблизить диагональ, oblique-дерево может провести одну наклонную линию и сразу аккуратно отделить рисунок от фона. Поэтому в приложении интереснее всего рисовать наклонные штрихи, дуги и буквы вроде V или Z: на них разница между моделями видна лучше всего.

6.2 Как это устроено внутри

При обучении oblique-дерева поиск разбиения меняется, а всё остальное, то есть построение узлов, листья и критерий Джини, остаётся прежним. Вместо одной координаты дерево перебирает набор направлений (a, b) . Часть из них фиксированная: горизонталь, вертикаль и две диагонали под 45° . Остальные генерируются случайными углами, а их количество задаёт ползунок **Направления для oblique**. Для каждого направления считается проекция точки:

$$z = ax + by,$$

и дальше дерево ищет порог c для z точно так же, как искало бы порог для обычной координаты x или y . Формула индекса Джини и критерий выигрыша остаются теми же.

Чем больше направлений, тем выше шанс найти удачный наклон, но и обучение идёт медленнее: для каждого узла приходится перебирать не одну координату, а десятки возможных углов. На ползунке в приложении диапазон от 4 до 40 направлений. Для маленьких аккуратных рисунков хватает и нижней границы, а для сложных форм есть смысл подкрутить значение повыше.

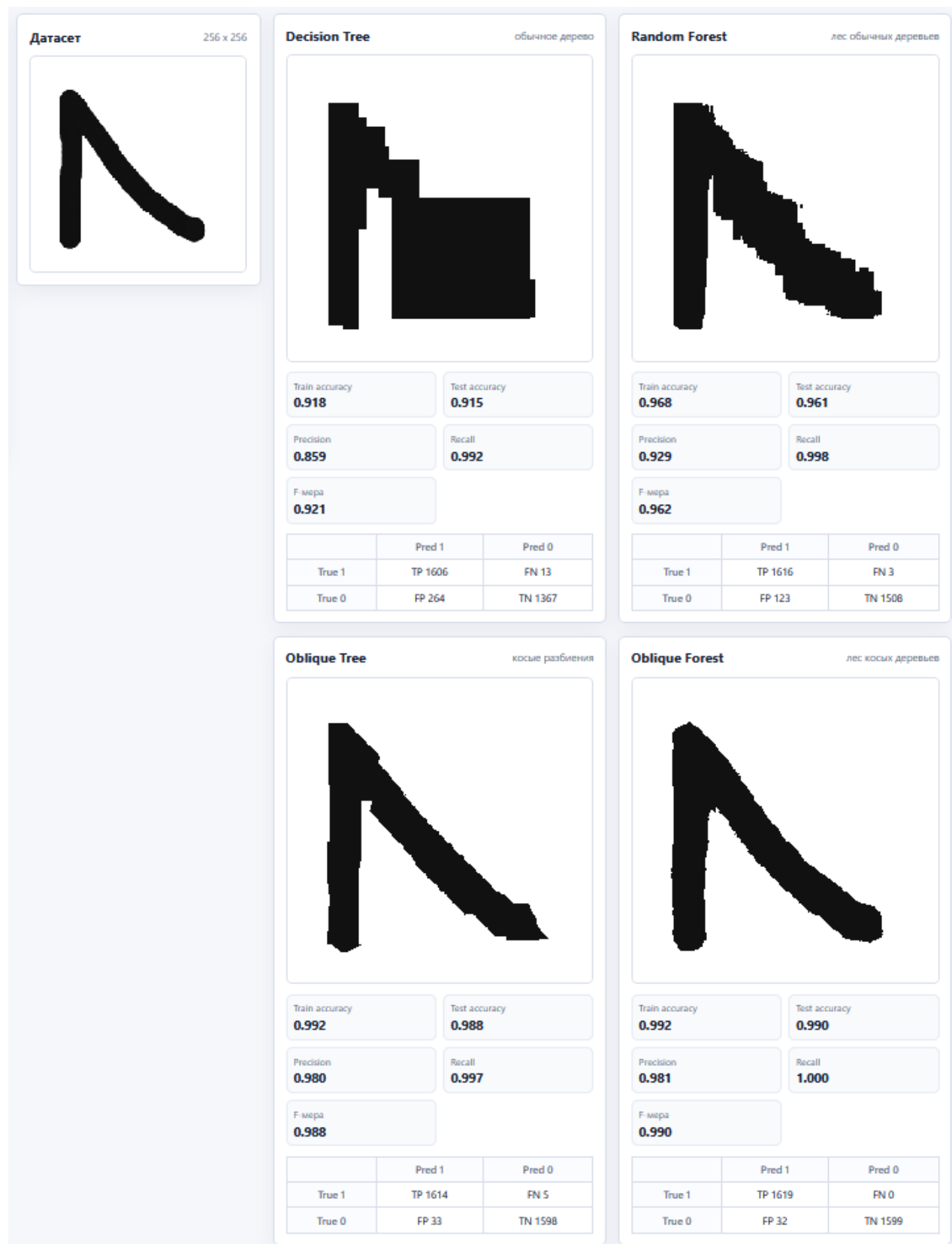


Рис. 4: Косые модели лучше работают с диагональными частями рисунка, потому что умеют делать наклонные разбиения.

7 Как устроено веб-приложение

Приложение сделано без фреймворков: только HTML, CSS и JavaScript. Это осознанный выбор: так проект открывается как обычная статическая страница, и в код легко заглянуть целиком, не разбираясь со сборкой.

Репозиторий: https://github.com/iosmoke1/rf_oblique_visualisation. Само приложение живёт по адресу https://iosmoke1.github.io/rf_oblique_visualisation/.

Файлы устроены так:

- `index.html`: разметка страницы, панель настроек, холст, шаблон карточки модели;
- `style.css`: внешний вид, сетка, карточки, таблица метрик;
- `app.js`: главный сценарий, который собирает настройки с ползунков, обучает модели и выводит результат;
- `src/drawing.js`: рисование на холсте и отрисовка предсказаний;
- `src/dataset.js`: превращение холста в train/test датасет;
- `src/tree.js`: дерево, лес и oblique-разбиения;
- `src/tree_view.js`: схема дерева и подсветка областей его узлов;
- `src/metrics.js`: метрики и confusion matrix;
- `model_demo.ipynb`: короткая демонстрация той же идеи через `sklearn`.

7.1 Путь данных внутри сайта

Вся цепочка выглядит так:

рисунок $\rightarrow$ пиксели $\rightarrow$ таблица $(x, y, label)$ $\rightarrow$ train/test $\rightarrow$ обучение моделей $\rightarrow$ результаты.

По нажатию кнопки **Обучить модели** `app.js` вызывает функцию `makeDataset`. Она читает пиксели `canvas` и превращает их в объекты вида $\{x, y, label\}$. Белого фона на любом рисунке почти всегда намного больше, чем чёрных линий, так что напрямую использовать все точки нельзя. Модель могла бы почти всегда отвечать «фон» и получать неплохую ассигасу, ничего толком не выучив. Поэтому классы уравниваются: из фона и рисунка берётся примерно одинаковое число точек. Если чёрных точек совсем мало (меньше 30), приложение прямо просит дорисовать что-нибудь ещё.

После обучения каждая модель проходится по всем пикселям холста заново и для каждого выдаёт число от 0 до 1, оценку принадлежности к рисунку. Если оценка не меньше 0,5, пиксель закрашивается чёрным, иначе остаётся белым. Справа получается не копия исходного холста, а новый рисунок, собранный по правилу модели.

Под настройками и холстом можно выбрать модель и посмотреть её дерево. Для леса доступен выбор отдельного дерева. Наведение на узел подсвечивает область, до которой доходят точки; два оттенка красного показывают стороны разбиения. Ветви можно сворачивать и раскрывать нажатием на узел. У леса подсветка отдельного дерева показывается поверх общего результата, а не вместо него.

Интерактивная визуализация Random and Oblique trees and forests

Обучить модели

Очистить

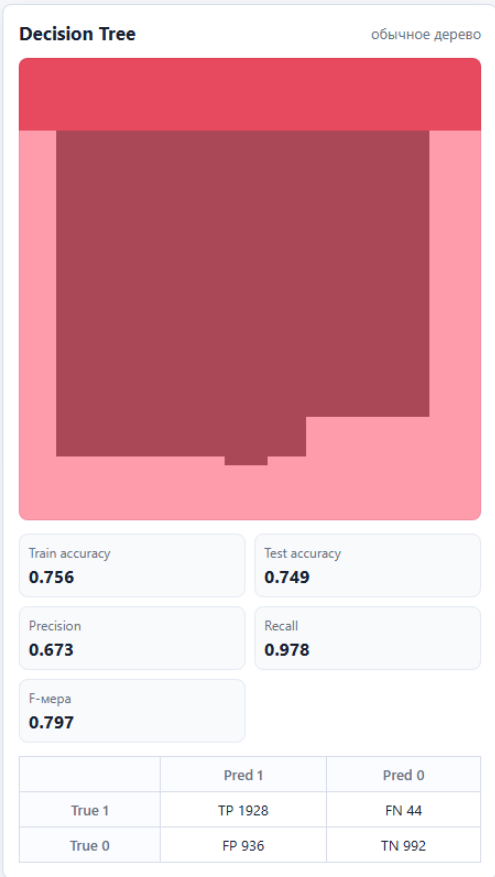
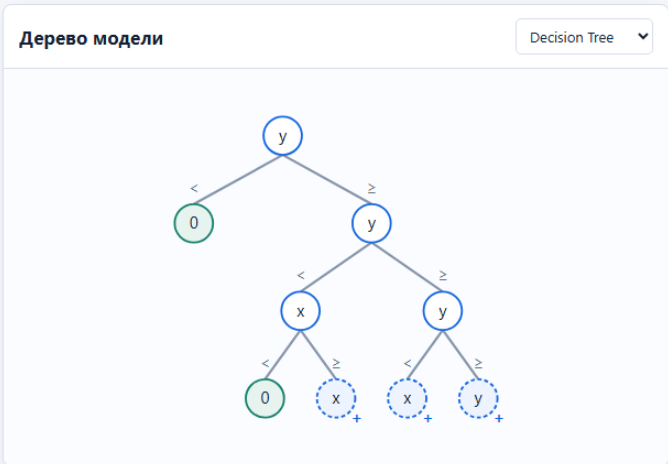


Рис. 5: Обычное дерево: при наведении на корень два оттенка красного показывают стороны разбиения.

Интерактивная визуализация Random and Oblique trees and forests

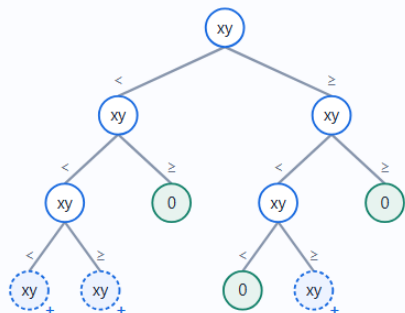
Обучить модели

Очистить

Дерево модели

Oblique Forest

Дерево 2



Датасет

256 x 256



Oblique Forest

лес косых деревьев



Train accuracy

0.956

Test accuracy

0.958

Precision

0.939

Recall

0.982

F-мера

0.960

	Pred 1	Pred 0
True 1	TP 1943	FN 36
True 0	FP 127	TN 1794

Рис. 6: Второе дерево oblique-леса: наклонное разбиение подсвечено поверх ответа всего леса.



Рис. 7: Метрики и confusion matrix помогают сравнить модели не только на глаз, но и числами.

8 Метрики качества

В приложении оставлены только те метрики, которые хорошо ложатся именно на задачу угадывания чёрных и белых пикселей.

Accuracy: доля всех правильно угаданных точек:

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Метрика понятная, но обманчивая: если фона на холсте намного больше рисунка, можно почти всегда угадывать «фон» и получать высокую ассигасу, хотя сама форма будет передана плохо. Именно из-за этого датасет и балансируется ещё на этапе подготовки данных.

Precision: какая доля закрасенных моделью точек и правда относится к рисунку:

$$precision = \frac{TP}{TP + FP}.$$

Низкий precision означает, что модель слишком щедро закрашивает лишний фон.

Recall: какую долю настоящего рисунка модель сумела найти:

$$recall = \frac{TP}{TP + FN}.$$

Низкий recall означает, что модель, наоборот, пропускает куски рисунка, недокрашивает.

F-мера собирает precision и recall в одно число:

$$F_1 = \frac{2 \cdot precision \cdot recall}{precision + recall}.$$

Она удобна, когда важно учитывать обе ошибки сразу: и лишнюю закраску, и пропуски.

Confusion matrix в карточках моделей это просто четыре числа в таблице:

	<i>Pred1</i>	<i>Pred0</i>
<i>True1</i>	<i>TP</i>	<i>FN</i>
<i>True0</i>	<i>FP</i>	<i>TN</i>

TP это верно найденный рисунок, *TN* это верно найденный фон, *FP* это лишняя закраска, *FN* это пропущенный кусок рисунка.

9 Jupyter Notebook

Отдельно к проекту прилагается тетрадка `model_demo.ipynb`. Её задача не в том, чтобы повторить JavaScript-код построчно. Она показывает, как ту же самую идею можно быстро собрать на готовых библиотеках Python.

В notebook то же самое сделано в разы компактнее:

- генерируется бинарная картинка на сетке 256×256 ;
- пиксели превращаются в таблицу признаков $(x, y, label)$;
- данные делятся на `train` и `test` через `train_test_split`;
- обучаются `DecisionTreeClassifier` и `RandomForestClassifier`;
- отдельным блоком показана идея oblique-разбиений: к признакам x и y добавляются проекции $ax + by$. После этого обычное дерево scikit-learn обучается уже на расширенной таблице признаков;
- считаются accuracy, precision, recall и F-мера для всех вариантов моделей;
- рисуются картинки, которые получились у моделей, и confusion matrix;
- отдельно показано, как применить уже обученную модель к новой точке (x, y) , которой не было в обучающей выборке.

Готового класса `ObliqueDecisionTreeClassifier` в `sklearn` нет, так что в notebook используется приближение через добавленные признаки, а не отдельная библиотечная реализация. Идею наклонных разбиений это, тем не менее, показывает вполне честно: тот же принцип, только собранный из подручных средств.

В sklearn нет готовой реализации косого решающего дерева. Поэтому здесь мы сами добавим признаки вида $a \cdot x + b \cdot y$

```
1 def add_oblique_features(X, n_directions=40):
2     # Каждое направление это проекция точки на наклонную ось
3     angles = np.linspace(0, np.pi, n_directions, endpoint=False)
4     directions = np.c_[np.cos(angles), np.sin(angles)]
5     projections = X @ directions.T
6     return np.c_[X, projections]
7
8 # Добавим новые признаки
9 X_train_oblique = add_oblique_features(X_train)
10 X_test_oblique = add_oblique_features(X_test)
11 X_all_oblique = add_oblique_features(X_all)
12
13 # Косые версии дерева и леса
14 oblique_tree = DecisionTreeClassifier(max_depth=7, min_samples_leaf=8, random_state=42)
15 oblique_forest = RandomForestClassifier(n_estimators=40, max_depth=7,
16                                       min_samples_leaf=8, bootstrap=True, random_state=42)
17
18 # Обучаем модели на новых признаках
19 oblique_tree.fit(X_train_oblique, y_train)
20 oblique_forest.fit(X_train_oblique, y_train);
```

Python

Рис. 8: Фрагмент Jupyter Notebook: добавление признаков $ax + by$ и обучение oblique-моделей.

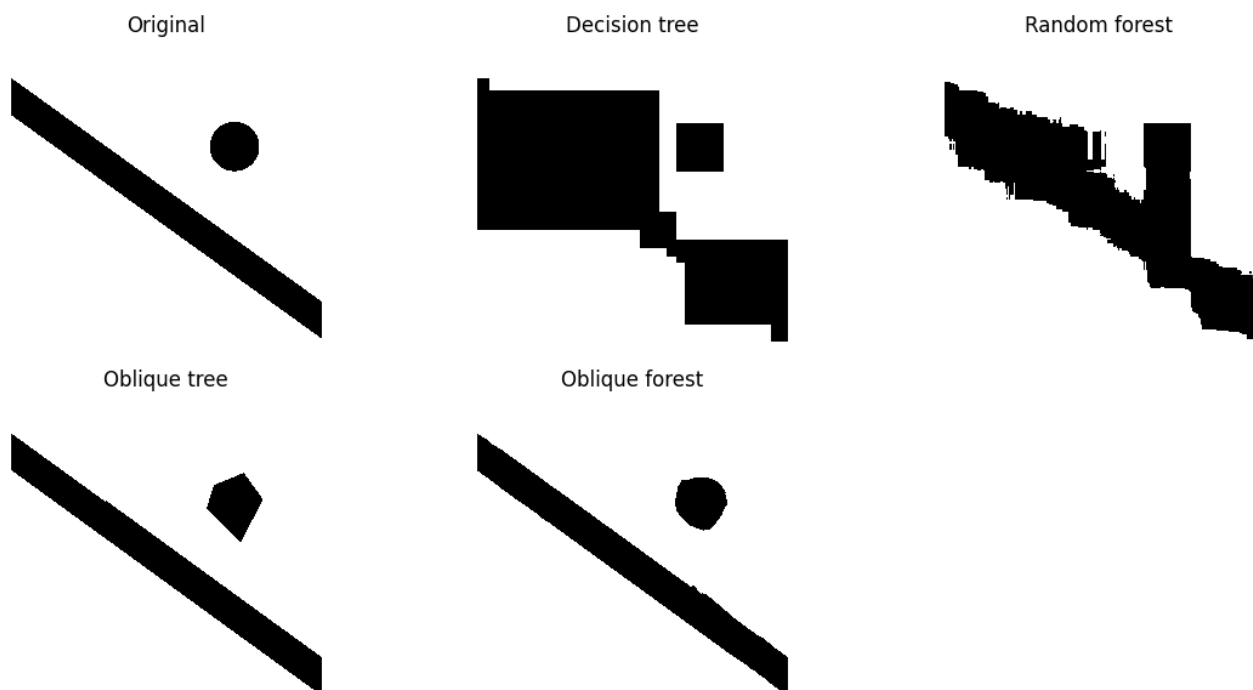


Рис. 9: Результаты notebook: исходная картинка и ответы разных моделей.

10 Как пользоваться материалами

Проще всего начать с веб-приложения: нарисовать что-нибудь несложное, например, диагональную линию, букву V или U, дугу, волну, квадрат, окружность, пару случайных штрихов, и нажать **Обучить модели**. Дальше остаётся сравнить четыре карточки справа. Обычно сразу видно, что одиночное дерево грубее остальных, лес выглядит стабильнее, а oblique-модели заметно лучше держат наклонные линии.

После этого стоит поиграть с ползунками. Маленькая глубина делает модель слишком грубой, большая добавляет деталей, но и шума заодно. Больше деревьев обычно делает лес устойчивее ценой скорости обучения. Больше направлений для oblique дают больше шансов на удачный наклон, но тоже стоят времени.

Дальше уже можно открыть `model_demo.ipynb`. Там та же идея, только через **sklearn**: создать данные, обучить модель, посчитать метрики, предсказать класс новой точки. Notebook хорошо связывает интерактивную картинку из браузера с обычным, «настоящим» ML-кодом.

11 Ограничения проекта

Проект учебный, и лишней инфраструктуры в нём намеренно нет: ни backend-сервера, ни сборщика, ни React, ни внешних JS-библиотек. Читать код так проще, но это же накладывает и ограничения.

Реализация деревьев не тягается по скорости с промышленными библиотеками. Она написана ровно настолько быстро, чтобы приложение отвечало за разумное время, не больше. Ещё качество результата сильно зависит от того, что именно нарисовал пользователь: на очень тонком или почти пустом рисунке точек класса 1 откровенно не хватает, и приложение об этом честно предупреждает.

Для учебного проекта такие ограничения нормальны. Цель здесь не в том, чтобы обогнать **scikit-learn** по скорости, а в том, чтобы после взаимодействия с приложением стало понятно: решающее дерево делит пространство прямыми углами, лес усредняет много таких деревьев, а oblique-дерево умеет поворачивать границу под нужным углом.

12 Заключение

Задача классификации здесь сведена к максимально наглядной форме: модель пытается повторить рисунок по координатам его точек. Именно поэтому разница между алгоритмами видна не в цифрах метрик, а прямо на картинке.

Обычное решающее дерево простое и легко объяснимое, но его возможности ограничены прямыми углами. Случайный лес убирает часть случайных ошибок одного дерева за счёт усреднения многих. Косые деревья добавляют более гибкий вопрос вида $ax + by < c$ и благодаря этому справляются с формами, которые обычному дереву даются с трудом.

Мне кажется, основная польза этого проекта в том, что алгоритм можно не просто прочитать в коде, а проверить своими руками: нарисовать форму, подвигать ползунки, посмотреть на результат, сравнить метрики, а потом открыть notebook и увидеть тот же самый процесс уже в привычном виде на Python.

13 Список литературы

Список литературы

- [1] Breiman L., Friedman J., Olshen R., Stone C. Classification and Regression Trees. Wadsworth, 1984.
- [2] Breiman L. Bagging predictors // Machine Learning. 1996. Vol. 24. No. 2. P. 123–140.
<https://link.springer.com/article/10.1007/BF00058655>
- [3] Breiman L. Random Forests // Machine Learning. 2001. Vol. 45. No. 1. P. 5–32.
<https://link.springer.com/article/10.1023/A:1010933404324>
- [4] Biau G., Scornet E. A random forest guided tour // Test. 2016. Vol. 25. No. 2. P. 197–227.
<https://link.springer.com/article/10.1007/s11749-016-0481-7>
- [5] Ishwaran H., Kogalur U. B. Consistency of random survival forests // Statistics & Probability Letters. 2010. Vol. 80. No. 13–14. P. 1056–1064.
<https://www.sciencedirect.com/science/article/pii/S0167715210000672>
- [6] Cutler A., Cutler D. R., Stevens J. R. Random forests // Ensemble Machine Learning. Springer, New York, NY, 2012. P. 157–175.
https://link.springer.com/chapter/10.1007/978-1-4419-9326-7_5
- [7] Murthy S. K., Kasif S., Salzberg S. A System for Induction of Oblique Decision Trees // Journal of Artificial Intelligence Research. 1994. Vol. 2. P. 1–32.
- [8] Random Forest explained. MLU-Explain.
<https://mlu-explain.github.io/random-forest/>
- [9] Scikit-learn documentation: Decision Trees and Random Forests.
<https://scikit-learn.org/stable/modules/tree.html>
<https://scikit-learn.org/stable/modules/ensemble.html#forest>